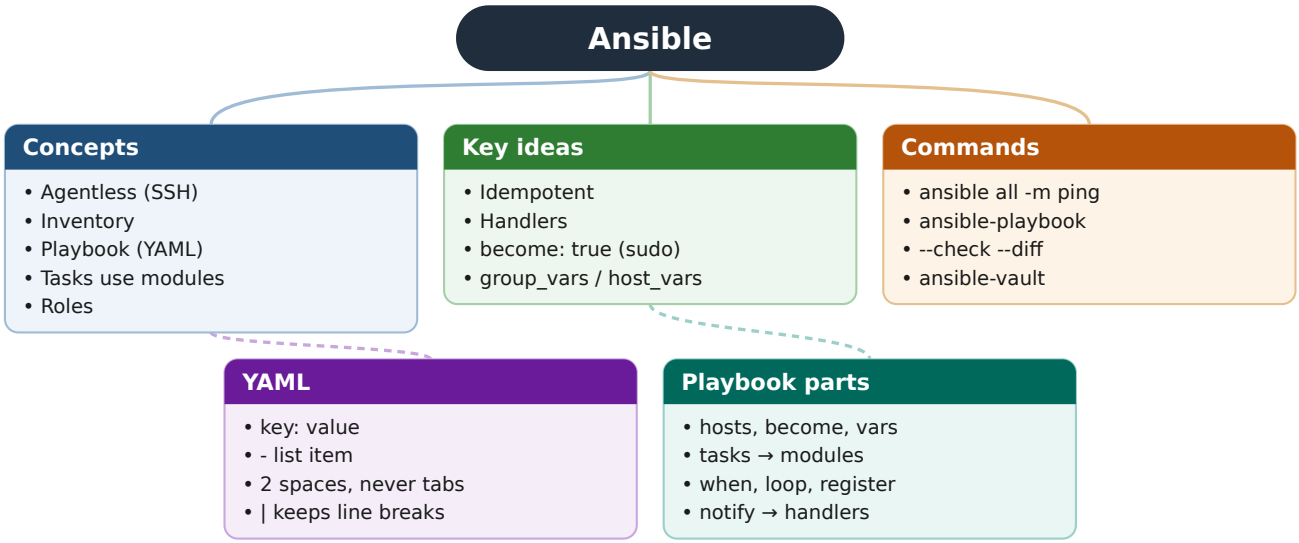
10. Ansible (Configuration Management)

DevOps, Cloud & Platform Engineering · Complete Notes · Topic 10 of 18 · Thirupathi Keloth · v2.0

Contents

10.0 Why configuration management
10.1 Concepts and architecture
10.2 Installation and ansible.cfg
10.3 Inventory
10.4 Ad-hoc commands
10.5 YAML for Ansible (and Kubernetes, Compose, Actions)
10.6 Playbook anatomy
10.7 Most-used modules
10.8 Variables, facts and precedence
10.9 Jinja2: templates, filters and tests
10.10 Controlling execution
10.11 Roles and collections
10.12 Secrets with Ansible Vault
10.13 Project layout and best practices
10.14 Testing and quality
10.15 Ansible in the bigger picture
10.16 Project: web server role
10.17 Troubleshooting
10.18 Interview questions and answers
10.19 Self-check: I can…



10.0 Why configuration management

Terraform creates servers; something must **configure** them: packages, users, files, services, security settings, application deployments. Doing that by hand over SSH on 50 servers is slow and produces servers that drift apart. **Configuration management** describes the desired configuration as code and applies it consistently and repeatably.

Where companies use Ansible: server configuration and hardening (CIS baselines), application deployment to VMs, patching, user management, network device configuration (Cisco, Juniper, Arista), cloud provisioning tasks, building golden images with Packer, orchestrating multi-step operations (rolling restarts).

10.1 Concepts and architecture

Concept	Meaning
Agentless	Nothing to install on targets: Ansible connects over **SSH** (Linux) or **WinRM/SSH** (Windows) and runs small Python modules there
Push model	The control node pushes changes when you run a playbook (Puppet/Chef classically **pull** via agents)
Idempotent	Modules describe a *state* ("package present"); running twice changes nothing the second time
Declarative-ish	Tasks run in order (procedural), but each module is declarative about its result

```
CONTROL NODE (your laptop / CI runner / AWX)
 ├── ansible.cfg            settings
 ├── inventory              which hosts, in which groups
 ├── playbooks (YAML)       what to do
 ├── roles / collections    reusable content and modules
 │
 └── SSH ──► managed node 1 (Python runs the module, returns JSON: ok / changed / failed)
     SSH ──► managed node 2
     SSH ──► managed node N   (in parallel: "forks")
```

Tool	Model	Language	Notes
Ansible	Agentless, push	YAML + Jinja2	Easiest start; most popular for config mgmt
Puppet	Agent, pull	Puppet DSL	Strong for large, long-lived fleets
Chef	Agent, pull	Ruby	Developer-oriented
Salt	Agent (or SSH), push/pull, event-driven	YAML + Jinja2	Very fast at scale
Terraform	Provisioning (API-driven)	HCL	Creates infrastructure; pairs with Ansible

Typical split: Terraform builds the VPC, instances, load balancer → Ansible configures the OS and deploys the app (or Ansible + Packer bake a golden AMI; containers often replace in-server configuration).

10.2 Installation and ansible.cfg

```
pipx install --include-deps ansible     # full package with community collections
# or a minimal core + only what you need:
pipx install ansible-core
ansible-galaxy collection install community.general ansible.posix amazon.aws
ansible --version                       # shows config file in use, Python, collection paths
```

Managed nodes need **Python 3** and SSH access (key-based) with a user that can `sudo`.

ansible.cfg is searched in this order (first found wins): `ANSIBLE_CONFIG` env var → `./ansible.cfg` (project directory) → `~/.ansible.cfg` → `/etc/ansible/ansible.cfg`. Keep one per project, committed to Git.

```
[defaults]
inventory = inventory/hosts.yml
remote_user = ubuntu
private_key_file = ~/.ssh/lab.pem
roles_path = roles
host_key_checking = True        ; keep True in production; False only for throwaway labs
forks = 20
stdout_callback = yaml
retry_files_enabled = False

[privilege_escalation]
become = False
become_method = sudo

[ssh_connection]
pipelining = True               ; fewer SSH round trips, faster runs
```

10.3 Inventory

The inventory lists managed hosts and organizes them into **groups**.

INI format

```
[web]
web1.example.com
web2.example.com ansible_host=10.0.1.21

[db]
db1.example.com ansible_user=ec2-user

[prod:children]
web
db

[web:vars]
http_port=80
```

YAML format (preferred for structure)

```
all:
  vars:
    ansible_user: ubuntu
  children:
    web:
      hosts:
        web1:
          ansible_host: 10.0.1.20
        web2:
          ansible_host: 10.0.1.21
      vars:
        http_port: 80
    db:
      hosts:
        db1:
          ansible_host: 10.0.21.10
    prod:
      children:
        web:
        db:
```

group_vars and host_vars

```
inventory/
├── hosts.yml
├── group_vars/
│   ├── all.yml          # variables for every host
│   ├── web.yml          # variables for group web
│   └── prod/
│       ├── vars.yml
│       └── vault.yml    # encrypted secrets (10.11)
└── host_vars/
    └── web1.yml         # variables for one host
```

Useful connection variables

`ansible_host` (IP/DNS to connect to) · `ansible_user` · `ansible_port` · `ansible_ssh_private_key_file` · `ansible_python_interpreter` · `ansible_connection` (`ssh`, `local`, `winrm`, `community.aws.aws_ssm`) · `ansible_become`.

Host patterns

Pattern	Hosts
`all`	Everything
`web`	Group web
`web:db`	Web **or** db
`web:&prod`	In web **and** prod
`prod:!db`	prod **except** db
`web1*`	Wildcard

Dynamic inventory (AWS)

Cloud hosts come and go, so build the inventory from the cloud API by tags:

```
# inventory/aws_ec2.yml
plugin: amazon.aws.aws_ec2
regions:
  - ap-south-1
filters:
  tag:Project: shop
  instance-state-name: running
keyed_groups:
  - key: tags.Role          # tag Role=web → group role_web
    prefix: role
  - key: tags.Env
    prefix: env
hostnames:
  - private-ip-address
compose:
  ansible_host: private_ip_address
```

```
ansible-inventory -i inventory/aws_ec2.yml --graph     # show groups and hosts
ansible-inventory -i inventory/ --list                 # full JSON with variables
```

10.4 Ad-hoc commands

One-off tasks without a playbook: `ansible <pattern> -m <module> -a "<args>"`.

```
ansible all -m ansible.builtin.ping                          # connectivity + Python check (not ICMP)
ansible web -m ansible.builtin.command -a "uptime"
ansible web -m ansible.builtin.shell -a "df -h | grep /dev/root"   # shell = pipes/redirects allowed
ansible web -b -m ansible.builtin.apt -a "name=htop state=present update_cache=true"
ansible web -b -m ansible.builtin.service -a "name=nginx state=restarted"
ansible web -m ansible.builtin.setup -a "filter=ansible_distribution*"   # facts
ansible db -m ansible.builtin.copy -a "src=./motd dest=/etc/motd" -b
ansible all -m ansible.builtin.ping -l web1 -f 10          # limit to web1, 10 forks
```

Flag	Meaning
`-i inventory`	Inventory file/directory
`-m module` / `-a "args"`	Module and its arguments
`-b` / `--become`	Run with sudo
`-K`	Ask for the sudo password
`-u user`, `--private-key`	Connection user/key
`-l pattern`	Limit hosts
`-f N`	Parallel forks
`-C` / `--check`, `-D` / `--diff`	Dry run / show changes
`-v`, `-vvv`, `-vvvv`	Verbosity (connection debugging)

`command` is safer than `shell` (no shell interpretation); both are **not idempotent**: prefer real modules.

10.5 YAML for Ansible (and Kubernetes, Compose, Actions)

YAML is used by Ansible, Kubernetes, Docker Compose, GitHub Actions and Helm, so learn it once.

```
# Comment
name: web-server          # key: value   (a space after the colon is required)
port: 8080                # number
enabled: true             # boolean (true/false)
version: "1.10"           # quote values that look like numbers but are text
empty: null               # no value

servers:                  # LIST: each item starts with "- "
  - web1
  - web2

database:                 # MAP (dictionary): nested keys, indented
  host: db.internal
  port: 5432

users:                    # list of maps
  - name: alice
    groups: [admin, dev]  # inline list
  - name: bob
    groups: [dev]

script: |                 # | keeps line breaks (multi-line text, scripts)
  echo "line 1"
  echo "line 2"
description: >            # > folds lines into one long line
  This is a long
  sentence.
```

Rules: indentation = structure, **spaces only (never tabs)**, usually 2 spaces per level, items in the same list must line up. Check a file with `yamllint file.yml` or `python3 -c "import yaml,sys; yaml.safe_load(open(sys.argv[1]))" file.yml`.

10.6 Playbook anatomy

```
---                                 # optional start-of-document marker
- name: Configure web servers       # PLAY: a name for humans
  hosts: web                        # which inventory group to run on
  become: true                      # use sudo for tasks
  vars:                             # VARIABLES for this play
    app_port: 8080
    packages:
      - nginx
      - curl

  tasks:                            # TASKS run in order, top to bottom
    - name: Install packages        # always name tasks: shown in the output
      ansible.builtin.apt:          # MODULE (fully qualified name)
        name: "{{ packages }}"      # {{ }} = Jinja2 variable (quote it)
        state: present              # desired state, not a command → idempotent
        update_cache: true

    - name: Deploy nginx site config
      ansible.builtin.template:     # renders a Jinja2 template file with variables
        src: templates/site.conf.j2
        dest: /etc/nginx/sites-available/default
        mode: "0644"
      notify: Reload nginx          # trigger the HANDLER only if this task changed something

    - name: Check if the app answers
      ansible.builtin.uri:
        url: "http://localhost:{{ app_port }}/health"
        status_code: 200
      register: health              # save the result in a variable
      ignore_errors: true

    - name: Show the result
      ansible.builtin.debug:
        msg: "Health status: {{ health.status | default('unknown') }}"   # | = Jinja2 filter

    - name: Create users
      ansible.builtin.user:
        name: "{{ item }}"
        groups: www-data
        append: true
      loop:                         # LOOP: run the task once per item
        - alice
        - bob

    - name: Only on Ubuntu
      ansible.builtin.debug:
        msg: "This is {{ ansible_facts['distribution'] }}"   # FACTS gathered from the host
      when: ansible_facts['distribution'] == "Ubuntu"         # CONDITION

  handlers:                         # HANDLERS run once at the end, only when notified
    - name: Reload nginx
      ansible.builtin.service:
        name: nginx
        state: reloaded
```

Template file `templates/site.conf.j2` (Jinja2 = text with `{{ variables }}` and `{% logic %}`):

```
server {
    listen 80;
    location / {
        proxy_pass http://127.0.0.1:{{ app_port }};
    }
}
```

Element	Meaning
Play	Maps a group of hosts to a list of tasks
Task	One step, using one module
Module	The tool that does the work (`apt` , `copy` , `template` , `service` , `user` , `file` , `uri` , `command`)
`state`	Desired result: `present` / `absent` , `started` / `stopped` / `restarted`
`{{ var }}`	Insert a variable (Jinja2)
`register`	Save a task's result
`when`	Run the task only if the condition is true
`loop` / `item`	Repeat a task for each item
`notify` / `handlers`	Run a follow-up (e.g. restart) only when something changed
`become`	Run with sudo
Facts (`ansible_facts`)	Information collected from each host (OS, IPs, memory)

Role layout (reusable unit, created with `ansible-galaxy init nginx`):

```
roles/nginx/
├── tasks/main.yml        # the tasks
├── handlers/main.yml     # handlers
├── templates/            # .j2 files
├── defaults/main.yml     # default variables (easy to override)
└── vars/main.yml         # fixed variables
```

Use it in a playbook with `roles: [nginx]` . **Avoid `command` / `shell` modules** when a real module exists: real modules are idempotent and report "changed" correctly.

10.7 Most-used modules

Always use **fully qualified collection names (FQCN)**, e.g. `ansible.builtin.copy` , to avoid ambiguity. Docs: `ansible-doc ansible.builtin.copy` (examples at the bottom).

Module	Purpose	Example arguments
`ansible.builtin.package` / `apt` / `dnf`	Install/remove packages	`name: [nginx, curl]` , `state: present/latest/absent` , `update_cache: true`
`ansible.builtin.service` / `systemd_service`	Start/stop/enable services	`name: nginx` , `state: started` , `enabled: true` , `daemon_reload: true`
`ansible.builtin.copy`	Copy a file or inline content	`src` , `dest` , `owner` , `group` , `mode: "0644"` , `content` , `backup: true`

Module	Purpose	Example arguments
`ansible.builtin.template`	Render a Jinja2 template	`src: app.conf.j2`, `dest`, `mode`, `validate: "nginx -t -c %s"`
`ansible.builtin.file`	Directories, links, permissions, delete	`path`, `state: directory/link/absent/touch`, `mode`, `owner`
`ansible.builtin.lineinfile`	Ensure one line in a file	`path`, `regexp`, `line`, `state`
`ansible.builtin.blockinfile`	Ensure a block of lines	`path`, `block`, `marker`
`ansible.builtin.user` / `group`	Accounts	`name`, `groups`, `append: true`, `shell`, `state`
`ansible.posix.authorized_key`	SSH public keys	`user`, `key: "{{ lookup('file', 'alice.pub') }}"`
`ansible.builtin.git`	Clone/update a repo	`repo`, `dest`, `version: v1.4.0`
`ansible.builtin.get_url` / `unarchive`	Download / extract	`url`, `dest`, `checksum`; `src`, `remote_src: true`
`ansible.builtin.uri`	HTTP requests/health checks	`url`, `status_code: 200`, `return_content`
`ansible.builtin.command` / `shell`	Run commands (last resort)	`cmd`, `creates:` / `removes:` to make them idempotent, `changed_when`
`ansible.builtin.cron`	Cron entries	`name`, `minute`, `hour`, `job`
`ansible.posix.mount`	Mounts + fstab	`path`, `src`, `fstype`, `state: mounted`
`ansible.posix.sysctl`	Kernel parameters	`name: net.ipv4.ip_forward`, `value: "1"`
`community.general.ufw` / `ansible.posix.firewalld`	Firewalls	`rule: allow`, `port: "443"`
`ansible.builtin.wait_for`	Wait for a port/file	`port: 8000`, `timeout: 60`
`ansible.builtin.reboot`	Reboot and wait	`reboot_timeout: 600`
`ansible.builtin.assert` / `fail`	Validate assumptions	`that: [...]`, `fail_msg`
`ansible.builtin.debug`	Print messages/variables	`msg`, `var`
`community.docker.docker_container`	Run containers	`name`, `image`, `ports`, `restart_policy`
`kubernetes.core.k8s`	Apply Kubernetes manifests	`state: present`, `src: deploy.yaml`
`amazon.aws.ec2_instance`, `amazon.aws.s3_object`	AWS resources	(prefer Terraform for long-lived infrastructure)

Return values

Every task returns JSON: `changed`, `failed`, `msg`, module-specific fields (`stdout`, `rc`, `status`…). Status colours in output: **ok** (green, nothing to do), **changed** (yellow), **failed** (red), **skipped** (cyan), **unreachable**. The play recap summarizes them per host.

10.8 Variables, facts and precedence

Where variables come from

Source	Example
Role `defaults/main.yml`	Lowest priority: sensible defaults users override
Inventory `group_vars` / `host_vars`	Per environment/group/host settings
Play `vars:` / `vars_files:`	Play-level values
`register:` / `set_fact:`	Values created during the run
Role `vars/main.yml`	High priority: internal constants of a role
Extra vars `-e "version=1.4.2"` / `-e @vars.yml`	**Highest priority**: always win

Simplified precedence (low → high): role defaults → inventory group_vars (`all` → specific groups) → host_vars → play vars / vars_files → role vars → block/task vars → set_fact/registered vars → **extra vars (`-e`)**. Rule of thumb: put user-tunable values in **role defaults** or **group_vars**; use `-e` only for one-off overrides.

Facts

Facts are gathered automatically from each host at the start of a play (`gather_facts: true`):

```
- name: Show some facts
  ansible.builtin.debug:
    msg: >-
      {{ inventory_hostname }} runs {{ ansible_facts['distribution'] }}
      {{ ansible_facts['distribution_version'] }} with
      {{ ansible_facts['memtotal_mb'] }} MB RAM, IP {{ ansible_facts['default_ipv4']['address'] }}
```

Disable with `gather_facts: false` for speed when you don't need them; cache them with fact caching (redis/jsonfile) on large fleets.

Magic variables

Variable	Meaning
`inventory_hostname`	Host name as in the inventory
`groups['web']`	List of hosts in group web
`hostvars['db1']['ansible_host']`	Variables of another host
`group_names`	Groups the current host belongs to
`ansible_play_hosts`	Hosts in the current play
`playbook_dir`, `role_path`	Paths

register and set_fact

```
- name: Check app version
  ansible.builtin.command: /opt/app/bin/app --version
  register: app_version
  changed_when: false               # a read-only command never "changes" anything

- name: Remember major version
  ansible.builtin.set_fact:
    app_major: "{{ app_version.stdout.split('.')[0] }}"
```

10.9 Jinja2: templates, filters and tests

Ansible evaluates `{{ expressions }}` and `{% statements %}` with **Jinja2**, in playbooks and `.j2` template files.

Filter	Example	Result
`default`	`{{ port | default(8080) }}`	Fallback value
`upper` / `lower` / `replace`	`{{ env | upper }}`	`PROD`
`join`	`{{ ['a','b'] | join(',') }}`	`a,b`
`to_json` / `to_nice_yaml` / `from_json`	Convert data	
`int` / `bool` / `string`	Type conversion	
`length`	`{{ servers | length }}`	Count
`map` / `select` / `selectattr`	`{{ users | selectattr('admin') | map(attribute='name') | list }}`	Filter lists of dicts
`regex_replace` / `regex_search`	`{{ url | regex_replace('^https?://', '') }}`	
`b64encode` / `password_hash('sha512')`	Encoding / password hashes	
`ipaddr` (ansible.utils)	Network calculations	

Template example `templates/nginx-site.conf.j2`:

```
# Managed by Ansible - manual changes will be overwritten
upstream app {
{% for host in groups['app'] %}
    server {{ hostvars[host]['ansible_host'] }}:{{ app_port }};
{% endfor %}
}

server {
    listen 80;
    server_name {{ server_name }};
{% if enable_gzip | default(true) %}
    gzip on;
{% endif %}
    location / {
        proxy_pass http://app;
    }
}
```

Lookups read data on the **control node**: `{{ lookup('env', 'HOME') }}`, `{{ lookup('file', 'files/key.pub') }}`, `{{ lookup('amazon.aws.aws_secret', 'shop/db') }}`, `{{ lookup('ansible.builtin.password', '/dev/null length=24') }}`.

10.10 Controlling execution

Conditionals and loops

```
- name: Install packages on Debian family only
  ansible.builtin.apt:
    name: "{{ item }}"
    state: present
  loop: [nginx, curl, jq]
  when: ansible_facts['os_family'] == "Debian"

- name: Create users with their groups
  ansible.builtin.user:
    name: "{{ item.name }}"
    groups: "{{ item.groups }}"
    append: true
  loop:
    - { name: alice, groups: "sudo,devops" }
    - { name: bob, groups: "devops" }
  loop_control:
    label: "{{ item.name }}"        # cleaner output
```

Blocks, error handling and handlers

```
- name: Deploy with rollback
  block:
    - name: Install new release
      ansible.builtin.unarchive:
        src: "releases/app-{{ version }}.tar.gz"
        dest: /opt/app
    - name: Health check
      ansible.builtin.uri:
        url: http://localhost:8000/health
        status_code: 200
      retries: 5
      delay: 3
      register: health
      until: health.status == 200
  rescue:
    - name: Roll back to previous release
      ansible.builtin.command: /opt/app/bin/rollback.sh
      changed_when: true
  always:
    - name: Report
      ansible.builtin.debug:
        msg: "Deployment of {{ version }} finished on {{ inventory_hostname }}"
```

Handlers run once at the end of the play, only if notified by a task that **changed**. Use `listen:` to group handlers; force them earlier with `- ansible.builtin.meta: flush_handlers`.

Rolling updates and orchestration

```
- name: Rolling deploy to web servers
  hosts: web
  become: true
  serial: 2                          # 2 hosts at a time (or "25%")
  max_fail_percentage: 0             # stop everything if any host fails
  pre_tasks:
    - name: Remove host from the load balancer
      ansible.builtin.debug:
        msg: "deregister {{ inventory_hostname }}"   # e.g. an AWS ELB module call
      delegate_to: localhost
  roles:
    - app
  post_tasks:
    - name: Add host back to the load balancer
      ansible.builtin.debug:
        msg: "register {{ inventory_hostname }}"
      delegate_to: localhost
```

Keyword	Meaning
`serial`	Batch size for rolling changes
`delegate_to: host`	Run this task on another host (e.g. localhost, a load balancer)
`run_once: true`	Run once for the whole batch (DB migration)
`tags: [nginx]`	Run/skip parts: `--tags nginx`, `--skip-tags slow`
`check_mode` / `diff` per task	Force dry-run or diff behaviour
`ignore_errors` / `failed_when` / `changed_when`	Control task status
`async` + `poll`	Long-running tasks in the background
`strategy: free`	Hosts proceed independently (default `linear` waits per task)
`throttle: 1`	Limit concurrency of one task

10.11 Roles and collections

Role structure

```
roles/nginx/
├── defaults/main.yml      # tunable defaults (lowest precedence)
├── vars/main.yml          # internal constants (high precedence)
├── tasks/main.yml         # entry point (can include other task files)
├── handlers/main.yml
├── templates/site.conf.j2
├── files/                 # static files for copy
├── meta/main.yml          # metadata, dependencies on other roles
└── README.md
```

`ansible-galaxy role init roles/nginx` creates the skeleton.

```
# site.yml
- name: Web tier
  hosts: web
  become: true
  roles:
    - role: common
    - role: nginx
      vars:
        server_name: shop.example.com
```

Other ways to use roles: `ansible.builtin.import_role` (static, at parse time) and `include_role` (dynamic, at runtime, can be in loops/conditions).

Collections and dependencies

Collections bundle modules, plugins and roles (`amazon.aws`, `community.general`, `kubernetes.core`, `cisco.ios`). Pin them in `requirements.yml`:

```
collections:
  - name: amazon.aws
    version: ">=8.0.0,<9.0.0"
  - name: community.general
    version: ">=9.0.0"
roles:
  - name: geerlingguy.docker
    version: "7.4.1"
```

`ansible-galaxy install -r requirements.yml` (also in CI). Ansible Galaxy hosts community content; review roles before trusting them.

10.12 Secrets with Ansible Vault

```
ansible-vault create group_vars/prod/vault.yml      # new encrypted file (opens $EDITOR)
ansible-vault edit group_vars/prod/vault.yml
ansible-vault view group_vars/prod/vault.yml
ansible-vault encrypt secrets.yml / decrypt secrets.yml
ansible-vault encrypt_string 'S3cr3t!' --name 'db_password'   # inline encrypted variable
ansible-playbook site.yml --ask-vault-pass
ansible-playbook site.yml --vault-password-file ~/.vault_pass  # file chmod 600, NOT in Git
ansible-playbook site.yml --vault-id prod@prompt --vault-id dev@~/.vault_dev
```

Pattern: `group_vars/prod/vars.yml` contains `db_password: "{{ vault_db_password }}"` and `vault.yml` (encrypted) contains `vault_db_password: ...`: you can grep variable names without decrypting. Mark tasks handling secrets with `no_log: true` so values don't appear in output/logs. Alternatively fetch secrets at runtime from **AWS Secrets Manager / HashiCorp Vault** lookups so nothing is stored in Git at all.

10.13 Project layout and best practices

```
ansible/
├── ansible.cfg
├── requirements.yml
├── inventory/
│   ├── dev/  (hosts.yml or aws_ec2.yml, group_vars/, host_vars/)
│   └── prod/
├── playbooks/
│   ├── site.yml          # everything
│   ├── web.yml
│   └── patch.yml
├── roles/
│   ├── common/           # users, SSH hardening, time sync, packages
│   ├── nginx/
│   └── app/
└── .ansible-lint
```

Best practices: idempotent tasks only (avoid `command` / `shell`, or use `creates` / `changed_when`) · name every task · FQCN modules · variables with role-name prefixes (`nginx_port`) · defaults in roles, environment differences in inventory · `--check --diff` before real runs · small, single-purpose roles · secrets in Vault/secret managers with `no_log` · pin collection/role versions · run from CI or AWX, not laptops, for production · keep playbooks in Git with reviews.

10.14 Testing and quality

Tool	Checks
`ansible-playbook site.yml --syntax-check`	YAML/playbook syntax
`yamllint .`	YAML style
ansible-lint	Best practices (FQCN, names, idempotency, risky shell usage)
`--check --diff`	Dry run showing what would change
Molecule	Role testing: creates test instances/containers, runs the role, runs it **again to verify idempotency**, verifies, destroys
CI	Lint + syntax + Molecule on every PR

10.15 Ansible in the bigger picture

Integration	How
Terraform + Ansible	Terraform outputs/tags → dynamic inventory → Ansible configures (or Terraform `local-exec` triggers a playbook)
Packer + Ansible	Ansible provisions a temporary VM → Packer saves it as a golden AMI → ASGs launch identical instances
AWS SSM	Connect through Session Manager (`community.aws.aws_ssm` connection) without SSH
Kubernetes	`kubernetes.core.k8s` / `helm` modules; but Kubernetes config is usually managed by GitOps (Argo CD)
AWX / Ansible Automation Platform	Web UI, RBAC, credentials store, schedules, job history, REST API

Integration	How
Semaphore UI, Rundeck	Lightweight alternatives
Network automation	`cisco.ios`, `arista.eos`, `junipernetworks.junos` collections (`network_cli` connection)

Performance tips: raise `forks`, enable `pipelining`, disable fact gathering when unneeded or cache facts, use `strategy: free` where order doesn't matter, avoid looping a module where it accepts a list (`apt name: [a, b, c]`).

10.16 Project: web server role

Goal: an idempotent role that installs nginx, deploys a site from a template, opens the firewall, and reloads nginx only when the config changes.

```
# roles/web/defaults/main.yml
web_server_name: "{{ inventory_hostname }}"
web_root: /var/www/site
web_port: 80
web_packages:
  - nginx
```

```
# roles/web/tasks/main.yml
- name: Install web packages
  ansible.builtin.package:
    name: "{{ web_packages }}"
    state: present

- name: Create web root
  ansible.builtin.file:
    path: "{{ web_root }}"
    state: directory
    owner: root
    group: root
    mode: "0755"

- name: Deploy index page
  ansible.builtin.template:
    src: index.html.j2
    dest: "{{ web_root }}/index.html"
    mode: "0644"

- name: Remove the distribution's default site (only our config should serve traffic)
  ansible.builtin.file:
    path: /etc/nginx/sites-enabled/default
    state: absent
  notify: Reload nginx

- name: Deploy nginx site configuration
  ansible.builtin.template:
    src: site.conf.j2
    dest: /etc/nginx/conf.d/site.conf
    mode: "0644"
  notify: Reload nginx

- name: Validate the complete nginx configuration
  ansible.builtin.command: nginx -t
  changed_when: false             # read-only check: never reports "changed"

- name: Ensure nginx is enabled and running
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: true
```

```
# roles/web/handlers/main.yml
- name: Reload nginx
  ansible.builtin.service:
    name: nginx
    state: reloaded
```

```
# roles/web/templates/site.conf.j2
server {
    listen {{ web_port }};
    server_name {{ web_server_name }};
    root {{ web_root }};
    index index.html;
}
```

```
# roles/web/templates/index.html.j2
<h1>{{ web_server_name }}</h1>
<p>Deployed by Ansible on
   {{ ansible_facts['distribution'] }} {{ ansible_facts['distribution_version'] }}</p>
```

```
# web.yml
- name: Configure web servers
  hosts: web
  become: true
  roles:
    - web
```

Run and verify idempotency:

```
ansible-playbook -i inventory/hosts.yml web.yml --check --diff   # preview
ansible-playbook -i inventory/hosts.yml web.yml                  # first run: changed > 0
ansible-playbook -i inventory/hosts.yml web.yml                  # second run: changed=0
curl http://<web-host>/
```

The second run reporting `changed=0` proves the role is idempotent.

10.17 Troubleshooting

Error	Cause	Fix
`UNREACHABLE! ... Permission denied (publickey)`	Wrong user/key, key not on host	Check `ansible_user`, `private_key_file`; `ssh -i key user@host` manually; `-vvvv`
`UNREACHABLE! ... Connection timed out`	Network/firewall/security group/bastion	Test SSH directly; configure `ProxyJump` in `ansible_ssh_common_args`
`Missing sudo password`	`become` needs a password	`-K`, or passwordless sudo for the automation user
`/usr/bin/python: not found`	Python missing or in another path	Install python3; set `ansible_python_interpreter=/usr/bin/python3`
`Could not import the python3-apt module`	Ansible uses a Python that lacks the OS apt bindings (e.g. a newer Python than the system one)	Set `ansible_python_interpreter` to the system Python (`/usr/bin/python3`), or install `python3-apt` for it
`couldn't resolve module/action`	Collection not installed	`ansible-galaxy collection install ...` / `requirements.yml`
`'x' is undefined`	Variable missing for that host/group	Check precedence, group membership (`ansible-inventory --host h`), typo; use `default()`
`Syntax Error while loading YAML`	Indentation, tabs, unquoted `{{` at value start, colons in strings	`--syntax-check`, yamllint, quote values
Task always shows `changed`	`command` / `shell` without `changed_when` / `creates`	Use proper modules or set `changed_when`
Handler didn't run	Notifying task didn't change, name mismatch, play failed before the end	Check names; `--force-handlers`; `meta: flush_handlers`
Very slow runs	Fact gathering, serial loops, low forks	Pipelining, more forks, fact caching, list arguments
Host key verification failed	New/changed host keys	Add to `known_hosts` (ssh-keyscan) or manage keys properly

Debug helpers: `-vvv`, `ansible.builtin.debug: var=...`, `ansible-inventory --host <h>` (all variables for a host), `ansible-playbook --step` (confirm each task), `--start-at-task "name"`.

10.18 Interview questions and answers

#	Question	Answer
1	What is Ansible and why agentless?	Configuration management tool that connects over SSH/WinRM and runs modules; no agents to install or maintain
2	Ansible vs Terraform?	Ansible configures systems (procedural tasks, no state file); Terraform provisions infrastructure declaratively with state
3	What is idempotency in Ansible?	Running a playbook again produces no changes if the system is already in the desired state
4	Inventory: static vs dynamic?	Fixed list of hosts vs generated from a cloud API by tags (aws_ec2 plugin)
5	Playbook vs play vs task vs role?	File of plays; a play maps hosts to tasks; a task calls a module; a role packages tasks/handlers/templates/vars
6	What are handlers?	Tasks triggered by `notify` from changed tasks, run once at the end (e.g. restart a service)
7	`command` vs `shell`?	`command` runs without a shell (safer); `shell` allows pipes/redirects; both not idempotent by default
8	How do you make a command task idempotent?	`creates` / `removes`, `changed_when`, or use a dedicated module
9	Explain variable precedence briefly	Role defaults lowest → inventory vars → play vars → role vars → set_fact/register → extra vars (`-e`) highest
10	What are facts?	System information gathered from hosts (OS, IPs, memory) available as `ansible_facts`
11	How do you handle secrets?	Ansible Vault (encrypted files/strings) or runtime lookups from secret managers, with `no_log`
12	How do you do a rolling update?	`serial`, `max_fail_percentage`, pre/post tasks with `delegate_to` for load balancer (de)registration
13	What is Jinja2 used for?	Templating variables, filters, loops and conditions in playbooks and config templates
14	`import_*` vs `include_*`?	Static (parsed upfront, tags inherited) vs dynamic (evaluated at runtime, can loop/condition)
15	How do you test roles?	ansible-lint, syntax-check, check/diff mode, Molecule (converge + idempotency test)
16	How do you run only part of a playbook?	Tags (`--tags` / `--skip-tags`), `--limit`, `--start-at-task`
17	What is `delegate_to`?	Runs a task on a different host than the current target (e.g. localhost or a load balancer)
18	What is AWX / AAP?	Web UI/API for Ansible: RBAC, credentials, schedules, job history
19	How do you speed up Ansible?	More forks, pipelining, fact caching or no facts, `strategy: free`, list arguments instead of loops
20	A host is unreachable. How do you debug?	`ansible host -m ping -vvvv`, manual SSH with the same user/key, network/SG/bastion checks

10.19 Self-check: I can…

- [ ] Explain configuration management, agentless push, idempotency and how Ansible complements Terraform.
- [ ] Install Ansible and collections; write a project `ansible.cfg`.
- [ ] Write static inventories (INI/YAML) with groups, group_vars/host_vars, and an AWS dynamic inventory.
- [ ] Run ad-hoc commands with the right flags.
- [ ] Write correct YAML and complete playbooks with handlers.
- [ ] Use the most common modules with FQCN and read return values.
- [ ] Explain variable precedence and use facts, magic variables, register and set_fact.
- [ ] Use Jinja2 filters, templates with loops/conditions, and lookups.
- [ ] Use loops, conditions, blocks/rescue/always, tags, delegate_to, serial rolling updates.
- [ ] Build roles, use collections and pin dependencies with requirements.yml.
- [ ] Protect secrets with Vault or secret manager lookups and `no_log`.
- [ ] Lint and test with ansible-lint, check/diff and Molecule.
- [ ] Build the web role and prove idempotency (`changed=0` on the second run).
- [ ] Troubleshoot connection, privilege, variable, YAML and idempotency problems.

> **REMEMBER**
>
> - **Terraform builds the house; Ansible furnishes it.**
> - Idempotent = running it twice changes nothing the second time.